

Anticipation Mechanism RISC-V Extension Proposal

January 28th, 2026

Introduction

This document's purpose is to propose an ISA for an anticipation mechanism: a hardware feature which anticipates the execution of select instructions, preempting execution to a handler (akin to hardware interrupts) *before* said instruction executes.

Motivation for Anticipation

This project is an extension of a previous work by Dr. Tessler: BUNDLE. BUNDLE seeks to efficiently schedule multithreaded applications in order to reuse the cache between threads. The theoretical benefits of BUNDLE rely upon a hypothetical **anticipation mechanism** that preempts the software thread by jumping to a handler address in anticipation of (before) the execution of certain instructions.

BUNDLE involves sectioning code belonging to a given process into *conflict-free regions* (CFRs) that can be run concurrently on the same core (by different software threads) without cache eviction penalties between executions after all relevant regions of memory for that CFR have been cached. The hypothetical anticipation mechanism allows for a process to share a cache across several threads that execute the same CFR concurrently. BUNDLE then proceeds to schedule subsequent CFRs.

The focus of this project is to implement the hardware support BUNDLE relies on: the anticipation mechanism. The anticipation mechanism enables CFRs to be dynamically specified by the scheduler. The purpose of hardware anticipation points (APs) is enabling a scheduler to block (preempt/deschedule) each software thread at the end of a given CFR, allowing execution of the same CFR by other (pending) threads, then proceeding to the next CFR. This allows software threads to share cached regions of memory over consecutive executions of a given CFR, theoretically reducing cache evictions and the associated cost of accessing main memory.

Brief

The proposed solution is an implementation of an anticipation mechanism as an extension of RISC-V. This solution would borrow from standard extensions to add new control/status registers (CSRs) and a new instruction for atomically returning execution to the thread such that the general purpose registers are in the same state as they were at the time of preemption. This project is intended to first establish a valid description of the modification to the RISC-V instruction set architecture (ISA), with initial testing through modification of Gem5, eventual hardware description language (HDL) implementation for verification on an FPGA, and to have manufactured a prototype very-large-scale integrated (VLSI) chip for verification of the BUNDLE scheduling algorithm.

Contents

The rest of this proposal will be broken into the following sections:

1. Background
2. Approach 1
3. Approach 2
4. Commentary on Design Decisions
5. Conclusion

Background:

The main interface for this anticipation mechanism involves using Control/Status Registers (CSRs), which are included in RISC-V as a standard extension called [Zicsr](#) [1, Ch. 6, p. 49]. This extension adds instructions to RISC-V for manipulating CSRs.

Here is a list of some relevant CSR instructions from the **Zicsr** extension:

CSRRW rd, csr, rs1

reads CSR, writing to rd register, and then reads rs1 register, and writes to csr.

CSRWS rd, csr, rs1

reads CSR, writing to rd register, and then reads rs1 register, and sets bits of CSR according to the bitmask in rs1.

CSRRC rd, csr, rs1

reads CSR, writing to rd register, and then reads rs1 register, and clears bits of CSR according to the bitmask in rs1.

CSRRWI rd, csr, uimm

reads CSR, writing to rd register, and then reads writes **uimm** to csr.

CSRRSI rd, csr, uimm

reads CSR, writing to rd register, and then sets bits of CSR according to the bitmask of **uimm**.

CSRRCI rd, csr, uimm

reads CSR, writing to rd register, and then clears bits of CSR according to the bitmask of **uimm**.

Zicsr also includes pseudo-instructions for common tasks like reading or writing to/from a CSR.

CSRR rd, csr -> CSRRS rd, csr, zero

reads csr, writing to rd register, without writing to csr.

CSRW csr, rs1 -> CSRRW x0, csr, rs1

reads from rs1, writing to csr without writing to rs1.

CSRWI csr, uimm -> CSRRWI x0, csr, uimm

writes (5-bit) immediate uimm to csr without writing to any register.

In addition to using CSRs, this proposed implementation of an anticipation mechanism also requires borrowing from the implementation of trap-return instructions. While **MRET** is included in the base ISA, **SRET** from the supervisor mode extension, and **URET** from the N mode extension should offer valuable insight for the direct implementation of the mechanism's CSRs and return instruction. The **sdtrig** extension may also prove valuable for the exact implementation of the mechanism's trigger and preemption features.

Approach 1: N-Mode Trap-Return

An intuitive solution would avoid building any new instructions for return, instead opting to use the preexisting atomic trap-return instructions. This would prevent the need to implement any new instructions, instead focusing on implementation of the anticipation mechanisms through reuse of the interrupt trap handlers for each usermode. This would involve using N-Mode user-level instructions for user-level, an extension which is considered defunct at present.

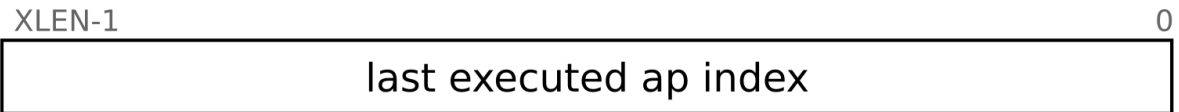
This solution involves adding a collection of new CSRs as listed below. Here, the N prefix refers to the N Mode extension for user-level interrupts.

Proposed CSRs:

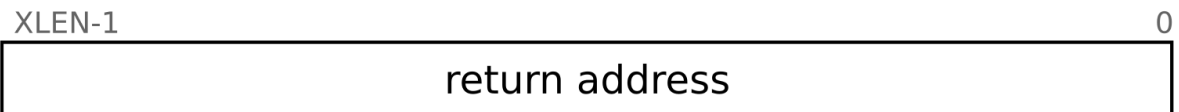
NAPSTATUS



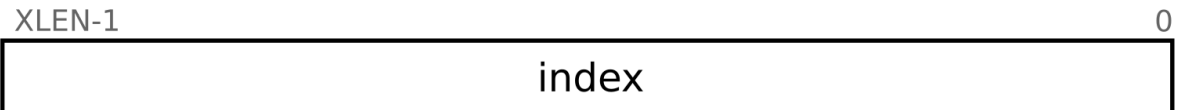
NAPLASTEX



NAPEPC

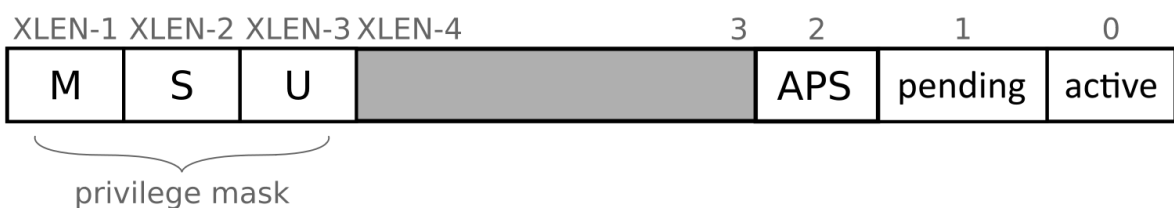


NAPSELECT

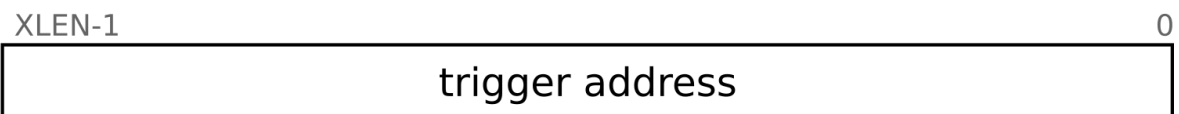


Selected by APSELECT:

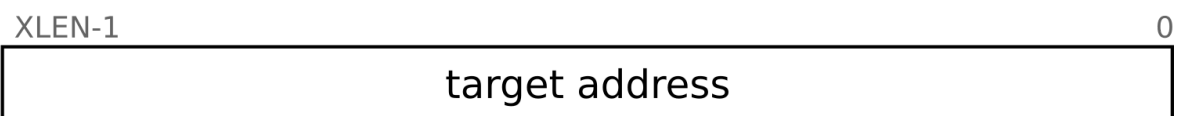
NAPCTRL



NAPTRIG



NAPTAR



NAPSTATUS (Anticipation Status):

This CSR holds relevant information for the anticipation mechanism as a whole.

APIE (Anticipation interrupt enable):

Enables anticipation points to preempt execution.

NAPLASTEX (Last Executed AP Index):

This CSR holds the index of the anticipation point that most recently caused a preemption. This is the same index as is used for **NAPSELECT**.

NAPEPC (Anticipation Exception PC):

This CSR holds the trigger address of the anticipation point which executed most recently.

NAPSELECT (Anticipation Point Select):

This CSR holds the index of the anticipation point which is currently selected. Subsequent reads and writes to **NAPCTRL**, **NAPTRIG**, and **NAPTAR** are directed to the hardware registers for the currently selected anticipation point.

NAPCTRL (Anticipation Point Control):

This CSR controls the behavior of the selected anticipation point.

Privilege Mask (M/S/U):

Determines the privilege levels (Machine, Supervisor, User) in which this anticipation point is active.

Pending:

A read-only bit indicating that this AP has been triggered but the handler has not yet been entered.

Active:

A read/write bit that enables or disables the selected anticipation point.

APS:

Anticipation point semaphore which bypasses anticipation point n times (this implementation uses one bit).

NAPTRIG (Anticipation Point Trigger):

This CSR holds the trigger address. When the program counter (PC) matches this address, and the AP is active, the anticipation mechanism is initiated.

NAPTAR (Anticipation Point Target):

This CSR holds the target address. When an anticipation event is triggered, the PC is set to this address, transferring execution to the handler.

Example Usage:

Here's an assembly (RV32) example of how the anticipation mechanism would function.

This first block sets up data and keywords for later use.

```
1  .data                                # Denotes a section reserved for data, rather than instructions.
2  #   exit_address                     # This is the address to which this example redirects URET,
3  #   such as to return to schedule another thread.
4  #   target_address                   # This is the address which begins the user defined handler
5  #   when anticipation takes place.
6  #   trigger_address                  # This is the address at which anticipation will occur.
7  .equ user_inactive, 0x20000000      # Here, the '2' represents a one in the U position of the
8  # privilege mask, with 0 in the 'active' flag
9  .equ user_active, 0x20000001        # Here, the '2' represents a one in the U position of the
10 # privilege mask, with 1 in the 'active' flag
11 .equ naps_mask 0x00000002           # This is the bitmask for the semaphore of the anticipation point.
12
13 size_of_array: .word 12              # The size of an array of integers.
14 array:          # The start of an array of integers.
15 .word 123, 456, 789
16 .word 123, 456, 789
17 boundary:      # This represents a boundary between different blocks of memory.
18 .word 123, 456, 789
19 .word 123, 456, 789
```

The next block sets up the anticipation point by writing to **napselect**, **napctrl**, **naptar**, and **naptrig**.

```

21 .text                                # Denotes a section reserved to instructions, rather than data.
22 .global _start                       # Standard way of telling the linker the baremetal code starts here.
23 _start:
24     csrw    napselect, zero           # Select anticipation point at index zero
25
26     li      t0, user_inactive
27     csrw    zero, napctrl, t0        # Set AP0 privilege level to user, and active to false
28
29     la      t0, target_address
30     csrrw   zero, naptar, t0         # Set AP0 target to target_address
31
32     la      t0, trigger_address
33     csrrw   zero, naptrig, t0        # Set AP0 trigger to trigger_address
34
35     li      t0, user_active
36     csrrsi  zero, napctrl, 1         # Set AP0 active to true

```

This next block serves as an example of the sort of code which might be preempted. In this case, a simple loop which iterates over a stored array, summing together the elements in the array.

```

38     li      a0, 12
39
40     la      t0, array                # This is the pointer to the current element of the array
41     lw      t1, size_of              # This is the counter
42     add     t2, zero, zero           # This is the current running total
43
44 for:
45     beq     t1, zero, end            # exit if counter==zero
46 trigger_address:
47     lw      t3, 0(t0)               # Load the currently pointed-to word from the array
48
49 ### Anticipation mechanism triggers ###
50
51     add     t2, t2, t3               # Add to running total
52     addi    t0, t0, 4                # Move pointer to next word
53     addi    t1, t1, -1               # decrement
54     j      for                      # continue the endless cycle.

```

Finally we have the handler to which execution is preempted upon the anticipation of execution of the instruction at the trigger address.

```

69 target_address:
70     # This is the address which begins the user-defined handler
71     # to which execution is preempted as a result of anticipation.
72 # For this example, execution branches to exit_address if the user is trying to load word from `boundary`
73
74     csrw    uscratch, t1             # Store the current value of t1 into uscratch
75
76     la      t1, boundary              # Load the address of the boundary into t1
77     bne     t0, t1, continue_execution # If the interrupted code isn't currently trying to load the word
78     # at boundary, then return execution.
79
80     # If the interrupted code *was* trying to load boundary, jump to exit_address
81
82     la      t1, exit_address          # Load the exit_address into t1
83     csrw    uepc, t1                 # Set the value of UEPC to the exit_address
84
85     csrr    t1, uscratch              # Before returning, we must load the initial value of t1
86     uret                                # MRET, SRET, URET, returns to address stored in xEPC
87
88 continue_execution:
89     csrr    t1, uscratch              # Return value of t1 from uscratch
90     csrsi   napstatus, naps_mask     # Set the semaphore bit of napstatus to `admit one` execution of the
91     # instruction at trigger_address.
92     uret                                # Return execution to interrupted instruction. This

```

On the first pass through the code which sums the elements of the array, preemption takes place on `lw t3, 0(t0)`.

```

37
38     li      a0, 12
39
40     la t0, array           # This is the pointer to the current element of the array
41     lw t1, size_of        # This is the counter
42     add t2, zero, zero    # This is the current running total
43
44 for:
45     beq t1, zero, end     # exit if counter==zero
46 trigger_address:         # This is the address of the instruction being anticipated.
47     lw t3, 0(t0)         # Load the currently pointed-to word from the array

```

In this implementation, when the PC reaches `trigger_address` and before executing the instruction at that address, a user-level interrupt triggers, atomically setting the UIP flag, resetting the UIE flag, storing the current PC to UEPC, setting PC to `target_address`, and setting `naplastex` to 0 (the index of the triggering anticipation point).

```

59 target_address:         # This is the address which begins the user-defined handler
60                         # to which execution is preempted as a result of anticipation.
61
62 # For this example, execution branches to exit_address if the user is trying to load word from `boundary`
63
64     csrw uscratch, t1    # Store the current value of t1 into uscratch
65
66     la t1, boundary      # Load the address of the boundary into t1
67     bne t0, t1, continue_execution # If the interrupted code isn't currently trying to load the word
68                                     # at boundary, then return execution.

```

Since the first pass doesn't pass the boundary, execution exits the handler and returns to the loop.

```

78 continue_execution:
79     csrr t1, uscratch    # Return value of t1 from uscratch
80     csrsi napstatus, naps_mask # Set the semaphore bit of napstatus to `admit one` execution of the
81                                     # instruction at trigger_address.
82     uret                 # Return execution to interrupted instruction. This

```

For this implementation, there is a need to determine when the anticipation mechanism should let *this one* pass. This can be done by using a bit[†] in `napctrl` as a semaphore which is atomically decremented/reset alongside the execution of the instruction at **trigger_address: NAPS** (N-mode AP semaphore).

[†] This implementation supposes one bit, but using more bits for **naps** may be preferable.

```

46 trigger_address:         # This is the address of the instruction being anticipated.
47     lw t3, 0(t0)         # Load the currently pointed-to word from the array

```

When returning (`uret`) to the **trigger_address**, since **NAPS** is set, the instruction executes, and NAPS is reset.

Approach 2: Independent Return (APRET)

An alternate approach involves adding a new instruction (APRET) which atomically resets the `pending` flag of APCTRL, sets the APIE of APSTATUS, and sets PC to the value stored in APEPC.

This solution involves adding a collection of new CSRs as listed below. Here, the **I** prefix refers to this mode being *independent* of interrupts and the associated trap-return implementation.

Proposed CSRs:

IAPSTATUS

XLEN-1

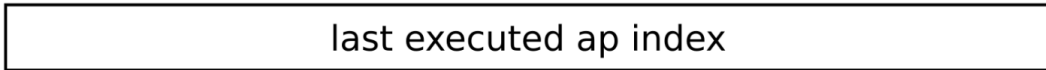
1 0



IAPLASTEX

XLEN-1

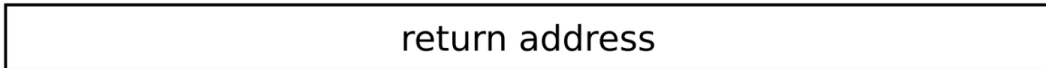
0



IAPEPC

XLEN-1

0



IAPSCRATCH

XLEN-1

0



IAPSELECT

XLEN-1

0



Selected by APSELECT:

IAPCTRL

XLEN-1 XLEN-2 XLEN-3 XLEN-4

3

2

1

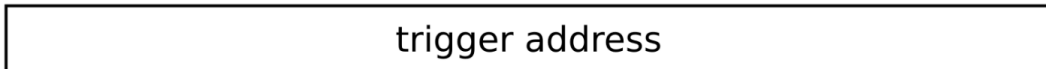
0



IAPTRIG

XLEN-1

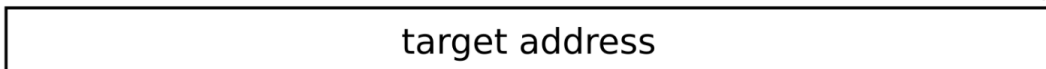
0



IAPTAR

XLEN-1

0



IAPSTATUS (Anticipation Status):

This CSR holds relevant information for the anticipation mechanism as a whole.

IAPIE (Anticipation interrupt enable):

Enables anticipation points to preempt execution.

IAPLASTEX (Last Executed AP Index):

This CSR holds the index of the anticipation point that most recently caused a preemption. This is the same index as is used for **IAPSELECT**.

IAPEPC (Anticipation Exception PC):

This CSR holds the trigger address of the anticipation point which executed most recently.

IAPSCRATCH (Independent AP Scratch):

A scratch register for use by the anticipation handler to save/restore state without using a general-purpose register.

IAPSELECT (Anticipation Point Select):

This CSR holds the index of the anticipation point which is currently selected. Subsequent reads and writes to **IAPCTRL**, **IAPTRIG**, and **IAPTAR** are directed to the hardware registers for the currently selected anticipation point.

IAPCTRL (Anticipation Point Control):

This CSR controls the behavior of the selected anticipation point.

Privilege Mask (M/S/U):

Determines the privilege levels (Machine, Supervisor, User) in which this anticipation point is active.

Pending:

A read-only bit indicating that this AP has been triggered but the handler has not yet been entered.

Active:

A read/write bit that enables or disables the selected anticipation point.

APS:

Anticipation point semaphore which bypasses anticipation point n times (this implementation uses one bit).

IAPTRIG (Anticipation Point Trigger):

This CSR holds the trigger address. When the program counter (PC) matches this address, and the AP is active, the anticipation mechanism is initiated.

IAPTAR (Anticipation Point Target):

This CSR holds the target address. When an anticipation event is triggered, the PC is set to this address, transferring execution to the handler.

Example Usage:

Here's an assembly (RV32) example of how the anticipation mechanism would function.

This first block sets up data and keywords for later use.

```
1  .data                                # Denotes a section reserved for data, rather than instructions.
2  #   exit_address                     # This is the address to which this example redirects URET,
3  #   such as to return to schedule another thread.
4  #   target_address                  # This is the address which begins the user defined handler
5  #   when anticipation takes place.
6  #   trigger_address                 # This is the address at which anticipation will occur.
7  .equ user_inactive, 0x20000000      # Here, the '2' represents a one in the U position of the
8  # privilege mask, with 0 in the 'active' flag
9  .equ user_active, 0x20000001       # Here, the '2' represents a one in the U position of the
10 # privilege mask, with 1 in the 'active' flag
11 .equ iaps_mask 0x00000002          # This is the bitmask for the semaphore of the anticipation point.
12
13 size_of_array: .word 12             # The size of an array of integers.
14 array:         # The start of an array of integers.
15 .word 123, 456, 789
16 .word 123, 456, 789
17 boundary:     # This represents a boundary between different blocks of memory.
18 .word 123, 456, 789
19 .word 123, 456, 789
```

The next block sets up the anticipation point by writing to **iapselect**, **iapctrl**, **iaptar**, and **iaptrig**.

```
21 .text                                # Denotes a section reserved to instructions, rather than data.
22 .global _start                       # Standard way of telling the linker the baremetal code starts here.
23 _start:
24     csrw   iapselect, zero            # Select anticipation point at index zero
25
26     li     t0, user_inactive
27     csrw   zero, iapctrl, t0          # Set AP0 privilege level to user, and active to false
28
29     la     t0, target_address
30     csrrw  zero, iaptar, t0           # Set AP0 target to target_address
31
32     la     t0, trigger_address
33     csrrw  zero, iaptrig, t0          # Set AP0 trigger to trigger_address
34
35     li     t0, user_active
36     csrrsi zero, iapctrl, 1           # Set AP0 active to true
37
38     li     a0, 12
```

This next block serves as an example of the sort of code which might be preempted. In this case, a simple loop which iterates over a stored array, summing together the elements in the array.

```

38     li      a0, 12
39
40     la t0, array           # This is the pointer to the current element of the array
41     lw t1, size_of        # This is the counter
42     add t2, zero, zero    # This is the current running total
43
44 for:
45     beq t1, zero, end     # exit if counter==zero
46 trigger_address:         # This is the address of the instruction being anticipated.
47     lw t3, 0(t0)         # Load the currently pointed-to word from the array
48
49 ### anticipation mechanism triggers ###
50
51     add t2, t2, t3        # Add to running total
52     addi t0, t0, 4        # Move pointer to next word
53     addi t1, t1, -1       # decrement
54     j for                # continue the endless cycle.
55
56 end:
57     j end                # Infinite loop to avoid executing miscellaneous data as code.

```

Finally we have the handler to which execution is preempted upon the anticipation of execution of the instruction at the trigger address.

```

59 target_address:         # This is the address which begins the user-defined handler
60                         # to which execution is preempted as a result of anticipation.
61
62 # For this example, we want to jump to our exit_address if the user is trying to load word from `boundary`
63
64 |   csrwr iapscratch, t1    # Store the current value of t1 into iapscratch
65
66     la t1, boundary        # Load the address of the boundary into t1
67     bne t0, t1, continue_execution # If the interrupted code isn't currently trying to load the word
68                                     # at boundary, then return execution.
69
70     # If the interrupted code *was* trying to load boundary, jump to exit_address
71
72     la t1 exit_address     # Load the exit_address into t1
73     csrwr iapepc, t1       #
74
75     csrr t1, iapscratch    # Before returning, we must load the initial value of t1
76     iapret                # Returns to address stored in IAPPC
77
78 continue_execution:
79     csrr t1, iapscratch    # Return value of t1 from iapscratch
80     csrsi iapstatus, iaps_mask # Set the semaphore bit of iapstatus to `admit one` execution of the
81                                     # instruction at trigger_address.
82     iapret                # Return execution to interrupted instruction.

```

On the first pass through the code which sums the elements of the array, preemption takes place on `lw t3, 0(t0)`.

```

38     li      a0, 12
39
40     la t0, array           # This is the pointer to the current element of the array
41     lw t1, size_of        # This is the counter
42     add t2, zero, zero    # This is the current running total
43
44 for:
45     beq t1, zero, end     # exit if counter==zero
46 trigger_address:         # This is the address of the instruction being anticipated.
47     lw t3, 0(t0)         # Load the currently pointed-to word from the array
48
49 ### anticipation mechanism triggers ###
50
51     add t2, t2, t3        # Add to running total
52     addi t0, t0, 4        # Move pointer to next word
53     addi t1, t1, -1       # decrement
54     j for                # continue the endless cycle.
55
56 end:
57     j end                # Infinite loop to avoid executing miscellaneous data as code.

```

In this implementation, when the PC reaches `trigger_address` and before executing the instruction at that address, the anticipation mechanism uses internal features to atomically set the *pending* flag of `IAPCTRL`, and reset the `IAPIC` of `IAPSTATUS`, store the current PC to `IAPEPC`, set PC to `IAPTAR`, and set `IAPLASTEX` to 0 (the index of the triggering anticipation point).

```

59 target_address:                # This is the address which begins the user-defined handler
60                                # to which execution is preempted as a result of anticipation.
61
62 # For this example, we want to jump to our exit_address if the user is trying to load word from `boundary`
63
64     csr w iapscratch, t1        # Store the current value of t1 into iapscratch
65
66     la t1, boundary             # Load the address of the boundary into t1
67     bne t0, t1, continue_execution # If the interrupted code isn't currently trying to load the word
68                                # at boundary, then return execution.

```

Since the first pass doesn't pass the boundary, execution exits the handler and returns to the loop.

```

78 continue_execution:
79     csrr t1, iapscratch         # Return value of t1 from iapscratch
80     csrsi iapstatus, iaps_mask # Set the semaphore bit of iapstatus to `admit one` execution of the
81                                # instruction at trigger_address.
82     iapret                     # Return execution to interrupted instruction.

```

For this implementation, there is a need to determine when the anticipation mechanism should let *this one* pass. This can be done by using a bit[†] in **iapctrl** as a semaphore which is atomically decremented/reset alongside the execution of the instruction at **trigger_address: IAPS** (independent AP semaphore).

[†] This implementation supposes one bit, but using more bits for **naps** may be preferable.

```

46 trigger_address:                # This is the address of the instruction being anticipated.
47     lw t3, 0(t0)                # Load the currently pointed-to word from the array

```

When returning (**iapret**) to the **trigger_address**, since **IAPS** is set, the instruction executes, and **IAPS** is reset.

```

51     add t2, t2, t3              # Add to running total
52     addi t0, t0, 4              # Move pointer to next word
53     addi t1, t1, -1             # decrement
54     j for                       # continue the endless cycle.

```

Once the loop tries to read from the boundary, the triggered handler alters the **IAPEPC** to return execution elsewhere, such as to a scheduler.

```

66     la t1, boundary             # Load the address of the boundary into t1
67     bne t0, t1, continue_execution # If the interrupted code isn't currently trying to load the word
68                                # at boundary, then return execution.
69
70 # If the interrupted code *was* trying to load boundary, jump to exit_address
71
72     la t1, exit_address         # Load the exit_address into t1
73     csr w iapepc, t1           #
74
75     csrr t1, iapscratch         # Before returning, we must load the initial value of t1
76     iapret                     # Returns to address stored in IAPEPC

```

Commentary on Design Decisions:

Why not just use hardware breakpoints?

Hardware breakpoints are limited in number for arm and x86, they allow their anticipated instruction to complete its pipeline before passing execution to the handler, and RISC-V doesn't offer a manner by which to manage hardware breakpoints from within (use of an external debugger is necessary to set and handle hardware breakpoints).

Why not just use ebreak?

The main reasons are equivalent to those of why we can't just use hardware breakpoints, particularly for RISC-V. Outside of user-level execution, a software mechanism approximates anticipation is achievable in **arm** and **x86** without hardware modification, however at higher cost in analysis and implementation than a dedicated hardware feature.

What's the proper way to store the context without incurring writeback penalty?

This is an essential future research task regarding BUNDLE, however cache locking appears to be integral for BUNDLE to see a positive cache benefit.

Why not mask apctr1 to include apselect?

If this is possible, it would reduce the number of new CSRs. However, the technical complexity and existence of precedent are so far left as a future research task.

Should we directly use URET from the N-Extension, Overhaul it, or copy only requisite features for Anticipation?

The N-mode extension was deprecated for a couple of reasons. The most significant reason is that the implementation was deemed unviable, and the consensus leading to its deprecation was that it should be refactored if it were brought back.

This being said, it's not worth refactoring the user-level interrupt extension as a subtask of this project, and as such it is most viable to copy the requisite features from interrupt handling features.

Conclusion:

This project is an extension of BUNDLE. The theoretical benefits of BUNDLE rely upon a hypothetical anticipation mechanism that preempts software threads by jumping to a handler in anticipation of the execution of specific instructions. The focus of this project is to implement the hardware support BUNDLE relies on: the anticipation mechanism. It is intended to first establish a valid description of the modification to the RISC-V instruction set architecture (ISA), with initial testing through modification of Gem5, and eventual hardware description language (HDL) implementation for verification on an FPGA. Ultimately, this project aims to offer a means by which to verify the efficacy of the BUNDLE scheduling algorithm since its execution depends upon an as yet nonexistent hardware feature.

Approach 1 uses the N-Mode extension for user-level interrupts, which are necessary for operation and configuration in user-level processes. Interrupt trap handling comes with an amount of overhead that is undesirable, and the N-mode extension is deprecated. Approach 2 avoids using interrupts altogether by implementing an internal mechanism of preemption based on interrupt trap handling, but not integrated into that system. This is a greater undertaking, but could result in a more cohesive extension. The viability of reducing the overhead compared with trap handling by using an internal mechanism remains a point of future research.

Approach 2 appears to be the better option. While this project is motivated by the need presented in BUNDLE, it is focused primarily on the anticipation mechanism itself, allowing for exploration of what an anticipation mechanism is without locking down architecture based on the specific needs of BUNDLE. The main drawback for approach 1 is the deprecation of the user-level interrupts extension, suggesting that future projects should avoid using it. This problem is avoided in approach 2 by implementing an internal return mechanism. Approach 2 offers a more stable foundation for future research and potential new system designs by avoiding deprecated extensions while enabling the implementation of the anticipation mechanism needed for BUNDLE.

References:

[1] Unprivileged Horizontal Committee, RISC-V International, [The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA](#), Version 20250508, May 2025.